

Helix Thready — Authentication & Authorization

Table of Contents

1. Scope & grounding
2. Credential types
3. JWT access + refresh
4. API keys with scopes
5. OAuth2 external linking
6. Three-tier RBAC
7. Scope catalog
8. MFA, sessions, passwords
9. Middleware chain (edge enforcement)
10. Gaps addressed
11. Open items

Rev	Date	Author	Change
1	2026-07-21	swarm (API & SDKs)	Initial draft grounded in <code>digital.vasic.auth</code>
2	2026-07-21	swarm (API & SDKs)	Documented the 4th credential path (HMAC <code>hmacAuth</code> for machine callbacks/webhooks); confirmed the 11-scope catalog is 1:1 with <code>openapi.yaml</code> ; linked <code>contract-tests.md</code>
3	2026-07-22	swarm (API & SDKs)	Depth pass, re-verified at source. Corrected the RS256 story: <code>jwt.Config.Secret</code> is <code>[]byte</code> used for BOTH sign+verify, so the module natively supports HMAC only — RS256/EdDSA is a module extension, and <code>manager.SetKey(...)</code> does not exist. Corrected <code>middleware.APIKeyHeader</code> signature (2nd arg is a header name, not a store). Added verified <code>pkg/oauth.AutoRefresher</code> defaults; the client-side <code>pkg/tokenmanager</code> refresh lifecycle; the raw-middleware error-body adaptation; <code>accesstoken.RevokeAllForUser</code> for logout-all. Closed <code>[OPEN: authz-2]</code> (kid scheme).

Table of Contents

1. Scope & grounding
2. Credential types
3. JWT access + refresh
4. API keys with scopes
5. OAuth2 external linking
6. Three-tier RBAC
7. Scope catalog
8. MFA, sessions, passwords
9. Middleware chain (edge enforcement)

10. Gaps addressed

11. Open items

1. Scope & grounding

Authentication and authorization are provided by `digital.vasic.auth` `[IN-HOUSE: auth]` `[VERIFIED]` (read at source: module `digital.vasic.auth`, `go 1.25.0`), composed into a new **User Service** `[GAP: New#User Service]` together with `security/pkg/policy` (enforcer) and the RBAC pattern proven in Catalogizer. This resolves Q9/Q10 and gap-register items `#10 / 7.2`.

The `auth` module packages actually present at source `[VERIFIED]`:

Package	Role in Thready
<code>pkg/jwt</code>	<code>Manager.Create/Validate/Refresh</code> , <code>Config.SigningMethod</code> — access+refresh tokens
<code>pkg/token</code>	<code>Token</code> interface, <code>Claims</code> map helpers, in-memory store with TTL + revocation
<code>pkg/apikey</code>	<code>APIKey{Scopes}</code> , <code>Generator.Generate(name, scopes, expiry)</code> , <code>Validate(store, key)</code> , <code>MaskKey</code> , <code>HasScope / HasAllScopes</code>
<code>pkg/oauth</code> + <code>pkg/oauth2provider</code>	OAuth2 credential management, refresh, caching, rate limiting
<code>pkg/middleware</code>	<code>BearerToken</code> , <code>APIKeyHeader</code> , <code>RequireScopes</code> , <code>Chain</code> , <code>ClaimsFromContext</code> , <code>ScopesFromContext</code>
<code>pkg/accesstoken</code> / <code>pkg/tokenmanager</code>	token lifecycle helpers
<code>pkg/i18n</code>	locale-aware error translation (NoopTranslator default; Thready injects HelixTranslate)

2. Credential types

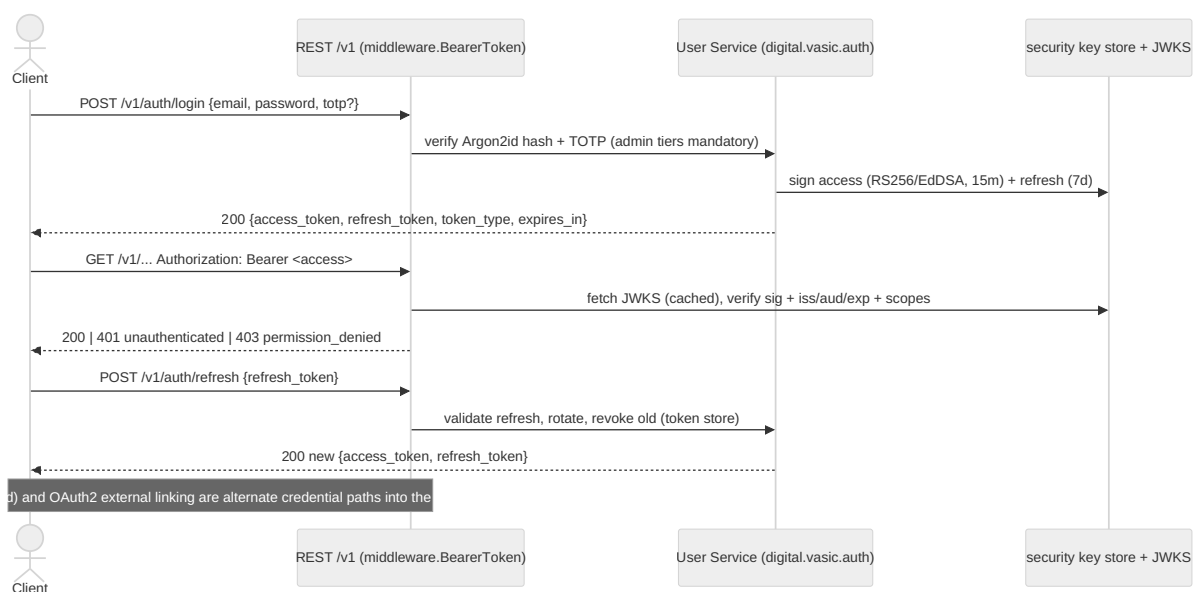
Three credential types authenticate against the **same** RBAC enforcer (Q10):

- **JWT (access + refresh)** — interactive clients (web, desktop, mobile, TUI login).
- **Scoped API keys (`sk-...`)** — SDK/CLI/automation (non-interactive).
- **OAuth2 authorization-code** — linking external services (Dropbox/GDrive/OneDrive), not primary user login.

In `openapi.yaml` these are the `bearerAuth`, `apiKeyAuth`, and `oauth2` security schemes. The default `security` requirement is `bearerAuth` OR `apiKeyAuth`; a handful of operations override it (`security:` `[]`) for `/auth/login`, `/auth/refresh`, `/.well-known/jwks.json`, `/healthz`, `/readyz`.

A **fourth, machine-only credential path** exists for server-to-server traffic that is not a user or an SDK: the `hmacAuth` scheme (`type: apiKey` , header `X-Thready-Signature` , an **HMAC-SHA256** over the raw body). It authenticates the **inbound 3rd-party callback** (`POST /v1/processing/callbacks/{provider}` — Boba/MeTube/Download Manager) and the **outbound `eventDelivery` webhook**. The shared secret is provisioned per provider/sink out of band and rotated via the secrets pack; a bad signature is `401 unauthenticated` . This path carries **no RBAC role** (`x-required-roles: []`) — trust is the secret, and the effect is scoped to the referenced `job_id` /sink. See `event-bus-contract.md` §9.

3. JWT access + refresh



Rendered PNG/SVG exported via Docs Chain (§11.4.65). Source: `diagrams/auth-flow.mmmd`.

Explanation (for readers/models that cannot see the diagram). A client posts its email, password and — for admin tiers — a TOTP code to `POST /v1/auth/login` . The User Service verifies the Argon2id password hash and the TOTP, then asks the `security` key store to mint two tokens: a short-lived access token (15 minutes) and a longer-lived refresh token (7 days). Both are returned as a `TokenPair` .

On every subsequent call the client sends `Authorization: Bearer <access_token>` ; the `middleware.BearerToken` validator fetches the JWKS (cached), verifies the signature and the standard claims (issuer, audience, expiry) plus the `scopes` claim, and either lets the request proceed (200), rejects an invalid/expired credential (401 `unauthenticated`), or rejects a credential lacking the required scope/role (403 `permission_denied`). Because the JWKS is cached with a short TTL and refetched on an unknown `kid` , verification adds no per-request network hop in steady state.

When the access token nears expiry the client calls `POST /v1/auth/refresh` ; the service validates the refresh token, issues a fresh pair, and **revokes the old refresh token** via the token store, giving rotation. API

keys and OAuth2 are alternate entry paths that resolve into the very same claims/scopes model, so authorization logic is written once — the note at the bottom of the diagram makes explicit that all three credential types converge on one enforcer.

Signing algorithm — the gap, verified precisely. [GAP: #10 / 7.2] [VERIFIED: pkg/jwt/jwt.go]

The `auth` module's `jwt.DefaultConfig` sets `SigningMethod: gojwt.SigningMethodHS256` (symmetric HMAC, `jwt.go:30`). HS256 is adequate for a single service that both signs and verifies, but Thready is a **multi-service** system (REST API, Event Bus service, Asset Service, callback ingress) where every service must *verify* tokens without holding the *signing* secret — so Thready needs **RS256 (or EdDSA)** asymmetric signing.

The gap is **deeper than a config flag**, and the earlier draft overstated the module's readiness. Read at source, `jwt.Config` carries a single `Secret []byte` field (`jwt.go:18`) that is used for **both** signing (`token.SignedString(m.config.Secret)`, `jwt.go:109`) **and** verification (the keyfunc returns `m.config.Secret`, `jwt.go:127`). That shape only supports symmetric algorithms: `gojwt`'s RS256/EdDSA signer requires a `*rsa.PrivateKey` / `ed25519.PrivateKey` for signing and the matching **public** key for verification, which a single `[]byte` cannot represent. There is also **no** `SetKey` method on `jwt.Manager` — its methods are exactly `NewManager`, `SetParser`, `Create`, `Validate`, `Refresh` (`jwt.go`). So RS256/EdDSA requires **one** of:

- `[BUILD-NEW]` extend `jwt.Config` in the `auth` module to accept `crypto.Signer` + a public-key verifier (a small, upstreamable change), **or**
- the **User Service** wraps `github.com/golang-jwt/jwt/v5` directly for the asymmetric sign/verify path and reuses the module only for the symmetric/internal cases.

Thready takes the first option (module extension, so every service keeps one auth dependency). The *contract* is unchanged either way — tokens are RS256/EdDSA, verified via JWKS:

```
// PROPOSED extended config (module change tracked as [BUILD-NEW]); illustrative.
cfg := jwt.Config{
    SigningMethod: gojwt.SigningMethodRS256,           // or EdDSA (Ed25519)
    Issuer:        "thready-user-service",
    Expiration:    15 * time.Minute,
    // Signer/Verifier added by the extension; private key loaded runtime-only
    // from a gitignored secret (CONST $11.4.10), never the []byte Secret.
}
manager := jwt.NewManager(&cfg)                      // Create() signs, Validate() verifies
// every OTHER service verifies against the PUBLIC JWKS at /.well-known/jwks.json
```

The public keys are published at `GET /v1/.well-known/jwks.json` (unauthenticated) so any service or SDK can verify tokens; **key rotation** publishes the new key with a fresh `kid` before retiring the old, and access tokens carry the `kid` header so verifiers pick the right key. Crucially, the algorithm-confusion protection is **already real in the module**: `jwt.Manager.Validate` pins the algorithm with `if t.Method.Alg() != m.config.SigningMethod.Alg() { return error }` (`jwt.go:122`) `[VERIFIED]`, so downgrade attacks (`alg: none`, HS256-signed-with-the-RSA- public-key) are rejected

regardless of which sign/verify wiring is chosen. That guarantee is what the `$security` negative controls in `contract-tests.md` assert.

Access-token claims (`[DEFAULT — adjustable]`):

```
{
  "iss": "thready-user-service",
  "sub": "ble2...user-uuid",
  "aud": "thready-api",
  "exp": 1721560800, "iat": 1721559900, "kid": "2026-07",
  "role": "account_admin",
  "account_id": "a1c2...account-uuid",
  "scopes": ["posts:read", "posts:write", "assets:read", "accounts:admin"]
}
```

4. API keys with scopes

Non-interactive clients mint keys at `POST /v1/api-keys` with an explicit scope list. Grounded on `pkg/apikey` `[VERIFIED]` :

```
gen := apikey.NewGenerator(&apikey.GeneratorConfig{Prefix: "sk-", Length: 32})
key, _ := gen.Generate("ci-bot", []string{"posts:read", "search:read"}, expiry)
// key.Scopes == ["posts:read","search:read"]; store.Store(key)
// On each request: apikey.Validate(store, presented) → *APIKey, then key.HasAllScopes(required)
// Logs/UI only ever show apikey.MaskKey(key) → "sk-ab...yz"
```

Rules:

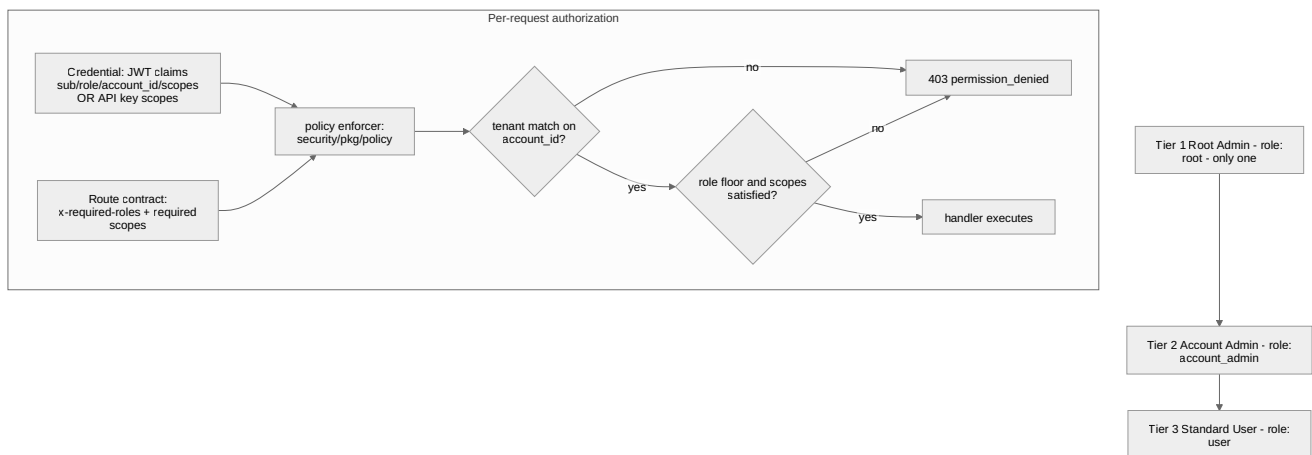
- The full secret is returned **once** at creation (`ApiKey.secret`), never again; only the masked form (`apikey.MaskKey`) is shown thereafter.
- Keys are presented as `Authorization: Bearer sk-...`; the `apiKeyAuth` scheme in `openapi.yaml`. Scope enforcement is identical to JWT scopes (`RequireScopes`).
- Keys carry an optional `expires_at`; expired/revoked keys fail `apikey.Validate`.
- A key's scopes are a **subset** of the minting principal's scopes — a `user` cannot mint a `root:admin` key (enforced server-side, returns 403).

5. OAuth2 external linking

OAuth2 (via `pkg/oauth` / `pkg/oauth2provider`) is used to **link external accounts** (cloud storage presets that also appear in `Auth-KMP`), not for primary login. Flow: `GET /v1/auth/oauth2/authorize?provider=dropbox&redirect_uri=...` → provider consent → callback exchanges the code for provider tokens, which are stored **encrypted** (AES-256-GCM via `security/pkg/securestorage`) against the user, and auto-refreshed with caching + rate limiting by the module. Provider tokens never enter Thready JWTs.

Verified refresh behaviour [VERIFIED: `pkg/oauth/oauth.go`]. The module's `oauth.AutoRefresher` (`NewAutoRefresher(reader, refresher, config, endpoints)`) proactively refreshes a provider credential when it is within a threshold of expiry, backed by a cache and a per-provider rate limit. `oauth.DefaultConfig()` sets `RefreshThreshold: 10m`, `CacheDuration: 5m`, `RateLimitInterval: 30s` (`oauth.go:271`), and `GetCredentials` returns the cached credential while fresh, re-reads on staleness, and only re-hits the provider token endpoint when `NeedsRefresh` is true and the rate-limit window has elapsed — so a burst of asset operations against a linked Dropbox account never stampedes the provider. If a token is **expired and refresh fails**, `GetCredentials` returns an error (the Asset Service surfaces this as `503 unavailable` for that link), rather than silently serving a dead token.

6. Three-tier RBAC



Rendered PNG/SVG exported via Docs Chain (§11.4.65). Source: `diagrams/rbac.mmd`.

Explanation (for readers/models that cannot see the diagram). Thready has exactly three roles matching the final request's §6.1 hierarchy. **Root Admin** (`role: root`, only one exists) has full control of every account and every user. **Account Admin** (`role: account_admin`) has full control of *their* account and its users. **Standard User** (`role: user`) has consumer access to the accounts they are members of. A user can belong to multiple accounts with a different role in each, captured by the `Membership` list.

On every request the enforcer (`security/pkg/policy`) receives two inputs: the credential (JWT claims — `sub`, `role`, `account_id`, `scopes` — or the API key's scopes) and the route contract (the operation's `x-required-roles` floor plus the required scopes). It first checks tenancy: the target resource's `account_id` must match the principal's account (or the principal must be `root`, which may cross tenants). If the tenant check fails, the request is denied with 403 `permission_denied` — and, per the ordering in `error-model.md` §4, this happens before any existence check, so a probe cannot distinguish “not yours” from “does not exist”.

If the tenant check passes, the enforcer checks that the principal's role meets the route's role floor **and** that the required scopes are all present; only then does the handler run. This keeps authorization declarative in the contract (`x-required-roles`) and enforced in one place in Go, never in the OpenAPI document itself — the contract *describes* the floor, the enforcer *imposes* it.

Role floor semantics. `x-required-roles: [user]` means “any authenticated principal whose role is at least `user`” — `account_admin` and `root` inherit everything a `user` can do within their scope; `[account_admin]` additionally requires admin of the target account (or `root`); `[root]` is root-only; `[]` is unauthenticated.

7. Scope catalog

Scopes are fine-grained, orthogonal to roles (`[DEFAULT – adjustable]` ; mirrors the OAuth2 scopes in `openapi.yaml`):

Scope	Grants
<code>posts:read</code>	read posts/threads/hashtags
<code>posts:write</code>	register channels, trigger (re)processing
<code>assets:read</code>	list/read/download assets
<code>assets:write</code>	manage assets, download jobs, re-download
<code>search:read</code>	run <code>/v1/search</code>
<code>skills:read</code>	read Skill-Graph
<code>skills:write</code>	register/update skills
<code>events:read</code>	subscribe to WS/SSE, read sticky snapshots
<code>accounts:admin</code>	administer an account + its users, branding, subscription
<code>billing:read</code>	read subscription/usage/invoices
<code>root:admin</code>	root-only system administration

A route requires **both** its role floor and its scope set; e.g. `POST /v1/skills` requires role `account_admin` **and** scope `skills:write` .

8. MFA, sessions, passwords

Resolves Q9 (`[DEFAULT – adjustable]` , tuned to the Aggressive posture):

- **MFA — TOTP mandatory for Root Admin and Account Admin**, optional for users. Enrolment via `/v1/auth/mfa/totp/enroll` → `/verify` .

- **Sessions** — access token **15 min**, refresh **7 d**, idle timeout **30 min** (web). Revocation through the `pkg/token/store (Store.Get/Set/Delete/Revoke, TTL + revocation, [VERIFIED: pkg/token/token.go])`. **Logout-all** (revoke every session for a compromised user) is backed by `accesstoken.MemoryStore.RevokeAllForUser(userID) int [VERIFIED: pkg/accesstoken]`, which the User Service exposes behind `POST /v1/auth/logout` with an `all: true` option. On the **client** side the SDK thin layer persists the pair via the `pkg/tokenmanager.Manager` shape (`StoreTokenInfo / GetAccessToken / IsExpired / ClearTokens, [VERIFIED: pkg/tokenmanager]`), which is exactly the R7 transparent-refresh recipe in `sdk-examples.md`.
- **Passwords** — Argon2id (`security`), **min 12 chars**, breach-list checked.
- **Secrets** — signing keys and OAuth tokens are runtime-loaded from gitignored `.env / secrets (chmod 600/700)`, never logged (`CONSTITUTION §11.4.10`).

9. Middleware chain (edge enforcement)

Enforcement happens in the shared chain (`pkg/middleware`), composed with `Chain(...)`, before any handler — so authz is uniform and un-bypassable:

```
secured := middleware.Chain(
    reqid.Middleware(),           // digital.vasic.middleware: request-id
    recovery.Middleware(),       // panic → 500 internal + trace_id
    cors.Middleware(corsCfg),    // CORS
    headers.Middleware(),        // security/pkg/headers
    ratelimiter.Middleware(rlCfg), // digital.vasic.ratelimiter → 429
    middleware.BearerToken(jwtValidator), // auth: TokenValidator → claims in ctx
    middleware.APIKeyHeader(apiKeyValidator, "Authorization"), // 2nd arg is the HEADER NAME
    middleware.RequireScopes("posts:read"), // per-route
)(handler)
// handler reads middleware.ClaimsFromContext(ctx) / ScopesFromContext(ctx) /
// APIKeyFromContext(ctx)
```

Verified signatures `[VERIFIED: pkg/middleware/middleware.go]`. The earlier draft passed a `store` as `APIKeyHeader`'s second argument; at source the signature is `APIKeyHeader(validator APIKeyValidator, headerName string) (middleware.go:103)` — the second argument is the **header name to read** (Thready reads `Authorization`, then the validator strips the `Bearer` prefix and calls `apikey.Validate(store, key)` internally). The two validator interfaces are minimal: `TokenValidator.ValidateToken(token) (map[string] interface{}, error)` and `APIKeyValidator.ValidateKey(key) ([]string, error)` — note the API-key path returns **scopes only, no role**, so a key's role floor is derived from the minting principal at creation time, not carried on the key. `BearerToken` also normalises the `scopes` claim through `coerceScopes`, which accepts both `[]string` and the `[]interface{}` form a JWT round-trip produces (`middleware.go:208`) `[VERIFIED]`.

Error-envelope adaptation. The raw module middleware writes terse bodies (`{"error": "invalid token"}` → 401, `{"error": "insufficient scopes"}` → 403; `middleware.go:78/166`), **not** Thready's

canonical envelope. The User Service therefore wraps the chain with a small adapter that rewrites these into the `error-model.md` envelope (`unauthenticated` / `permission_denied` with a `trace_id`), so clients see one error shape. `RequireScopes` (and the `security/pkg/policy` enforcer for role/tenant) gate the route; rate limiting sits ahead of auth so anonymous floods are shed cheaply (`[GAP: #14]` DDoS/rate limiting).

10. Gaps addressed

- `[GAP: #10 / 7.2]` JWT default HMAC-SHA256 → **RS256/EdDSA + JWKS rotation** (multi-service verification) — §3. Verified at source that HS256 is the module default and that `Validate` pins the algorithm.
- `[GAP: 7.2]` **No built-in RBAC in `auth`** → a reusable RBAC layer (root/account_admin/user) folded into the new **User Service** on `security/pkg/policy` — §6. `[GAP: New#User Service]`.
- `[GAP: 7.2]` **TOTP MFA for admin tiers** — §8.
- `[GAP: 7.1 security]` “encrypted yet semantically searchable credentials” — the searchable-but-sealed representation is specified in the architecture/database packs; the API never returns raw secrets (masked keys, encrypted OAuth tokens) — §4, §5.
- `[GAP: 7.3/7.4 Security-KMP]` mobile secure storage is an **in-memory stub** — the API contract is unaffected, but SDK guidance (mobile) **must** block release until native Keychain/KeyStore lands; cross-referenced in `sdk-strategy.md`.
- **TDD** — the security negative controls for every claim here (algorithm-confusion `alg:none` / HS256-with-public-key rejection, cross-tenant `403`, API-key scope-subset enforcement, admin-tier TOTP mandatory, HMAC bad-signature `401`) are written RED-first in `contract-tests.md` §security `[CONSTITUTION §11.4.27]`.

11. Open items

- `[OPEN: authz-1]` The exact `security/pkg/policy` rule DSL for tenant+role+scope composition is finalized with the User Service implementation; this doc fixes the *contract* (claims, scopes, role floors), not the enforcer’s internal rule syntax.
- `[RESOLVED: authz-2]` **JWKS rotation cadence + `kid` scheme fixed** (`[DEFAULT — adjustable]`): `kid = "<YYYY-MM>-<8-hex-of-thumbprint>"` (e.g. `2026-07-a1b2c3d4`) so the `kid` is human-legible in logs yet collision-safe. Rotation is **monthly**, overlap-first: the new key is published to `/.well-known/jwks.json` and used for signing on day 1 of the window, the previous key stays in the JWKS (verify-only) for one full access-token lifetime plus a 24 h margin, then is retired. Verifiers cache the JWKS with a short TTL and refetch on an unknown `kid`. Compromise triggers an out-of-band emergency rotation that also calls `RevokeAllForUser` for affected principals. The exact secrets-pack mechanics (HSM vs sealed file) remain deployment-pack detail; the **verifiable contract** (kid format, overlap, emergency path) is fixed here.

- `[BUILD-NEW: auth-rs256]` Extend `digital.vasic.auth/pkg/jwt.Config` to carry asymmetric key material (`crypto.Signer` + public verifier) so RS256/EdDSA is first-class in the module — see §3. Tracked as an upstreamable module change; the User Service ships with a direct- `gojwt` fallback until it lands so the contract (RS256 tokens + JWKS) holds on day one.
-

Made with love ♥ by Helix Development.